When Do Language Servers Help Coding Agents?

A Preprint

Ian Barber

July 2026

Abstract

We tested three language-server services against a text-capable baseline: symbol-to-definition resolution, a compact definition span in place of a whole file, and type-diagnostic timing.

Semantic navigation was near-neutral because the model reads the receiver's type in visible source and self-localizes. It helped only in a constructed regime that withheld that source, and only where the agent was framed to use it; merely making the operation available changed nothing. The compact span came from a static definition-span tool, not a live server. It cut total tokens 3.3–4.1x against whole-file retrieval and less against competent grep plus ranged reads, but only where it replaced the read. Pushing, instructing and self-election did not produce that replacement; training did, and produced a conditional policy rather than a blanket one — the trained model still read every time the span did not contain the defect. On twelve seeded defects, a submission-boundary checker moved accepted-correct outcomes from 1/12 to 11/12; the same checker after every edit changed nothing, on a channel that fired in one of twelve.

1. What we were investigating

A coding agent already has grep, ranged reads and a shell, so the useful question is not whether a language server can answer questions about code but whether it answers ones the agent cannot cheaply answer itself, in a cheaper form, or at a better moment.

IF: whether delivering the semantic information changes what the agent finds. A yes is the semantic arm resolving targets the text arm misses; a no is both arms resolving the same tasks at the same cost.

FORM: whether a compact span costs less than reading the file. A yes is total tokens falling at unchanged success; a no is the span arriving as context on top of the reads it was meant to replace.

WHEN: whether the moment a diagnostic arrives changes the outcome. A yes is the same diagnostic changing the accepted result depending only on when it fires; a no is the outcome flat across delivery phases.

Each is a claim tool builders make. Typed Holes [1] and LSPRAG [2] push types and definitions into the context on the premise that the information is missing; the first question tests that premise. STALL+ [3] reports static-analysis value varying with integration phase; the third tests that. Neither line reports a counterfactual that already reads code well, so the baseline here is a capable text agent.

2. How it was tested

Every result but one comes from a purpose-built harness: a model drives a fixed six-action loop — grep, ranged read, whole-file read, line edit, run tests, submit — over a Python workspace, with one semantic operation added or withheld per arm; the exception is one out-of-harness probe on real repositories. Nothing ran through an MCP server, a skill or an editor plugin.

Dispatch: fifteen tasks where a typed receiver calls one of about ten same-named overrides, one buggy, across a visibility ladder (L0 call-site annotation, L1 test construction site, L2 factory indirection) and a hidden rung withholding `app.py` and the test source, receiver construction redacted. `effic` and `effic_real2`: retrieval-cost tasks, `effic_real2` constructed misuse over vendored real library source (toolz 1.1.0, more-

itertools 11.1.0). navigation-v2: mechanically validated definition-span instances. checker-gate-v3: twelve seeded defect/clean pairs, each defect coherent, visible-passing and held-out-failing with one target-scoped semantic diagnostic, each clean control its validated gold. The other suites, `effic` included, are constructed from scratch: the real-repository scan cleared no admissible candidate (C17).

The efficiency thread's definition operation is a static AST resolver (`skip_pyrefly=True`, `ast.parse`), not a live language server; live Pyrefly ran only in dispatch goto arms and one 7B run.

Temperature 0, one rollout per cell: the dispatch ladder, navigation-v2, the paired retrieval run and checker-gate-v3. Temperature 0.7, repeated seeds: the hidden rung and its visible control (five seeds per task, 450 rollouts), the whole-file contrast, the election and execution-feedback runs (four seeds behind the 44-attempt cells), and the substitution harvest, with its held-out retest at 0. Models: Qwen2.5-Coder-7B, Qwen3.5-27B, Qwen3.6-27B locally; Sonnet 4.5, DeepSeek v3.1, GLM-4.6, GPT-5.6 "Luna" via API. Unit of analysis: the task, with task-level bootstrap intervals.

Per-claim configuration and artifacts: ledger, manifest, protocols.

3. Results

3.1 Does delivering the semantic information change what the agent finds?

On the annotated rung, grep and ranged reads resolved 15/15, neutral goto 14/15 and framed goto 15/15, at matched-success ratios 0.972 and 1.041 (C9). Across L0, L1 and L2, mean input tokens on resolved tasks were 1,436, 1,429 and 1,465 at 14–15/15, goto ratios 0.945–1.065 at every rung including L2, where only the server statically resolves the type. Trajectory counters: about 0.3 greps per task, the receiver's type read where it appeared, the buggy override opened first.

The hidden rung, against a matched visible control (C30):

Arm	Hidden	Paired difference vs grep	Greps	Whole-file reads	Visible
grep_base	70/75	—	1.52	2.17	75/75
defn_avail	71/75	+0.013 [−0.040, +0.067]	1.47	2.04	75/75
defn_prompt	75/75	+0.067 [+0.013, +0.133]	0.36	0.68	75/75

Per-task resolution rates over five seeds at temperature 0.7, 95% task bootstrap intervals over the fifteen tasks; every arm resolved every rollout in the visible control, so the effect appears only where the type was withheld.

Framed lookup improved four tasks (`codec_serialize`, `job_priority`, `resource_cost`, `row_format_row`) and worsened none; the same operation merely available improved three and worsened two. Cost landed on resolution, not tokens: matched-success input-token ratios were 0.983 framed and 1.005 available. The constructed hidden regime withholds source a real agent could usually open.

Typed lookup mechanically reached the exact gold override among 8–15 same-named overrides where erased lookup returned a non-discriminating base, both variants type-clean (C15). The two-task agent arm passed all 12 task-condition cells at a typed automatic/baseline total-token ratio of 1.037 (0.988–1.093), too wide for equivalence; every automatic result was followed by a target-file read, and composed lookup added about six seconds per task (C24).

3.2 Does a compact span cost less than reading the file?

On the retrieval-cost tasks, compact spans cut total tokens 3.35x, 3.49x and 4.12x against whole-file retrieval for Qwen3.6-27B, Sonnet 4.5 and DeepSeek v3.1, 44/44 resolved in every arm (C1).

Against grep plus ranged reads with whole-file fallback, eleven `effic_real2` tasks with Qwen3.5-27B cost 1,602 mean total tokens with text against 1,235 with definitions, a paired ratio of 1.297 (task bootstrap 1.093–1.527), cheaper on 10/11 at 11/11 success in both arms, the one suite and model where the span replaced the read, with 0 of 11 definitions followed by a reread (C27).

Untrained, the 7B elected the definition operation in 1 of 48 rollouts; relabel-trained, 48 of 48 with no file reads, mean input 3,086 to 687, resolution 31/48 to 48/48 (C2). Framing raised codenav use on two of three real-repository tasks at one seed (C3, C6).

With `lsp_defn` on in both `effic` arms, the untrained 7B invoked the definition tool 0/24 and the trained arm 24/24, mean input 2,894 to 689, success 14/24 to 24/24. The adapter is the election mechanism. A live-first Pyrefly backend with historical AST fallback reproduces the saving once the policy elects, backend unrecorded per row. No arm held the policy fixed while varying the backend, so this is not a measurement of a language-server token effect (C4).

Pushed spans were reread 35/36, and 34/36 with an explicit sufficiency instruction that removed 2/36 and added 294 mean tokens for the 27B (C31).

Model	Reread, pushed span	Reread, plus sufficiency instruction
Qwen3.6-27B	11/12	12/12
Sonnet 4.5	12/12	12/12
DeepSeek v3.1	12/12	10/12

A separate run with Qwen3.6-27B, GLM-4.6 and GPT-5.6 Luna reproduced the pushed reread at 35/36. Among instances each model elected, reread was 9/9, 4/9 and 7/12; GLM-4.6 solved only 5–6/12 of these tasks, and reread counted independently of success (C34).

In a bash agent (mini-swe-agent, Sonnet 4.5) on real repositories, roughly 16 of 18 `codenav defn` calls were followed by a read of the file just resolved, sympy's `str.py` at 22 `sed` invocations. Three tasks, one seed, 60-step cap, from committed summaries. That agent took 0–3 whole-file reads in 44–60 actions (C6, log).

A DAgger-style relabel [4] run on 39 demonstrations, none read-required, cut the post-span reread 11/12 to 0/12 on held-out instances disjoint in seed and template, mean input 1,157 to 748, a matched-success saving of 1.59x. Held-out pass moved 11/12 to 10/12: one rescue against two `xor` losses that pass visible and fail held-out, not distinguishable from noise at n=12 and one seed (C33). A pre-registered confirmation on twelve reserved instances, disjoint from the harvest in both seeds and templates and never run before, reproduced it: reread 12/12 untrained against 1/12 trained (11 removed, 1 persisted, 0 induced), mean input 1,223 to 729, a matched-success saving of 1.725x, and held-out pass 12/12 to 11/12 (C35).

What the training produced is a conditional policy, not suppression. Twelve further instances hoist each override's arithmetic into a module-level helper, so the span live Pyrefly returns for the call site is the complete definition of the method that binds and the bug sits outside it; a byte-paired twin differing in one line restores sufficiency, and a third arm leaves the repo identical and delivers a second genuine lookup at the helper (C36):

Reads after span	Span insufficient	Span sufficient	Helper also delivered
Untrained	12/12	12/12	11/12
Trained	12/12	2/12	0/12

Both conditions repaired 12/12 in every arm, visible and held-out, so this is retrieval behaviour at fixed correctness. The trained model read a mean of 1.00 times where reading was required against the untrained model's 3.42, and 0.17 and 0.00 times where it was not against 1.58 and 1.75; mean input tokens were lower in every arm (1,224 against 1,490; 924 against 1,158; 950 against 1,181). An adversary over seventeen one-parameter repair forms found none that reached held-out without retrieval, and the harness's post-edit file view was redacted because it would otherwise have handed over the hidden line with no read.

3.3 Does the moment the diagnostic arrives change the outcome?

Accepted type-clean and held-out-correct outcomes were 1/12 under control, 10/12 with one-shot diagnostics at revision and 11/12 behind a submission gate, for task-bootstrap effects against control of +0.375 [+0.250, +0.500] and +0.417 [+0.292, +0.500]. After-every-edit stayed at 1/12, +0.000 [−0.125, +0.125], on a channel that fired in 1 of 12 defect rows because the model edited before submitting in only 1 of 12 (C32).

Twelve seeded defect/clean pairs	Control	After every edit	One-shot	Gate
Bad completion accepted	11/12	11/12	2/12	1/12
Revision tokens, defect	787	754	1,210	1,380
Wall time, defect (s)	27.8	14.5	68.3	62.7
Revision tokens, clean	591	—	619	591
Wall time, clean (s)	9.4	9.9	9.6	9.5

After-every-edit fired in 1 of 12 defect rows; mean edits 0.08 against 0.17 in control.

The gate matched control on clean work in tokens and wall time, and spent about 2.2x control's wall time on defective work. It rejected 10 of 12 submissions, all 10 repairing, retesting and resubmitting, and 2 of 12 self-repaired beforehand; on `auth_pipeline_handler` the model received the diagnostic and submitted anyway with zero edits, so the gate's one-task edge over revision came from refusal, not information. Its single miss, `auth_shapes_protocol`, was type-clean after self-repair and failed held-out. The 0/12 false rejections are an apparatus check, since each clean control is that defect's validated gold and the checker is deterministic; population false-rejection rate is unmeasured. Calibration produced no usable natural checker-detectable defect: frontier inference sat at 18/18 and a descriptive 27B authoring arm at 12/12, 0/3 natural 7B drafts were coherent, and 2/8 coherent 14B drafts were already type-clean, so these twelve defects were seeded (C10, C11, C16, C22). A separate execution-feedback grid (two frontier models, 14 tasks, three seeds, three delivery modes) passed 252/252 (C12).

4. Findings

4.1 Delivery

On a fifteen-task constructed dispatch suite at a single seed, one rollout per cell, semantic navigation added little over grep and ranged reads: a capable model reads the receiver's type wherever it is visible and opens the right file itself, so the lookup delivers what the agent already holds. Cost stayed flat as the type moved from a call-site annotation to the test's construction site to factory indirection: what carries resolution is correct types in the source, not the server that queries them. The exception is a constructed regime withholding the type-bearing source, where framed lookup resolved every rollout over fifteen tasks at five seeds and text did not; the same operation merely available changed nothing, so what the server adds is contingent on election. Sound types also sharpen the resolver mechanically, but that precision bought no agent-level outcome in a two-task pilot.

4.2 Form

A compact span is cheaper than a read only when it replaces the read, and whether it does is a property of the agent's policy, not the tool. The span is cheaper by a wide margin against whole-file retrieval and by a narrower one against competent grep plus ranged reads. Election is separable: framing changed which operation the capable models called, and training changed it on a 7B. Substitution did not follow: on twelve instances per model at one seed, a pushed span was reread by a local 27B and by four frontier API models in two independent runs, and neither a sufficiency instruction nor self-election removed it. In a three-task, single-seed probe with a frontier model on sympy, astropy and sphinx, definition calls sat on top of reads; the whole-file read the largest ratios are measured against was rare there, so the achievable regime is the smaller one. Relabel training removed the reread where instruction and election could not, on one model at one seed and again on a pre-registered confirmation set built from templates the training never saw. What it trained is conditional rather than blanket: on twelve instances whose delivered span is genuine but does not contain the defect, the trained model read every time, and once rather than the untrained model's three times, while reading on two of twelve when the span sufficed and none of twelve when the missing definition was itself delivered. Correctness sat at ceiling in all six cells, so what training changed is when the agent retrieves, not whether it succeeds — and it learned that from demonstrations none of which required a read.

4.3 Timing

On twelve seeded defect/clean pairs, one model, one seed, a checker at the submission boundary converted accepted bad completions into repaired ones and one-shot delivery at revision performed comparably, while

delivering the same checker after every edit changed nothing, on a channel that fired in one of twelve defect rows. Late delivery works; early delivery is unmeasured. The submission gate is preferable to delivery at revision because it charges its cost only to defective submissions: clean work cost no more than control in tokens or wall time, and the price fell on wall time when a submission was defective. A type-clean gate is only as behaviorally sound as its checker. Natural prevalence is unmeasured, and the same ceiling that forced seeding left execution feedback nothing to move on the tested small-task suite.

5. If you use a coding agent

On a typed Python codebase, the clearest return is a type check at the submission boundary with a repair-and-resubmit loop.

Lever	Do this	Changes if
Do nothing	Add no semantic navigation tool over typed source the agent already opens (C9, C30)	Your source hides receiver types
Add a tool or MCP server	Add one only where source the agent opens hides the receiver's type — generated stubs, vendored or compiled boundaries — and frame its use (C30, C27)	You measure targets the agent misses
Skill or prompt change	Tell the agent which operation to use and when, and verify post-span reads before budgeting a saving (C3, C30, C31, C34)	A prompt removes the post-span read
Wire a hook	Gate submission on the checker, requiring repair and retest before acceptance; after-every-edit firing is untested (C32)	Your checker covers behavior, or your model edits repeatedly
Fine-tune	Fine-tune only with weights you control; it buys a conditional policy — the trained model still read every time the span did not contain the defect, and once rather than three times (C33, C36)	Your insufficiency looks unlike a delegation

Keep type annotations correct and present; the text baseline exploited exactly that. On your own workload, measure wrong-file edits, post-span reads of the defining file and spurious gate rejections; the clean-work cost here came from workspaces guaranteed clean. No result here ranks MCP servers, skill files and project instructions as delivery surfaces.

6. Future work

Every estimate rests on constructed workspaces; the closest real-repository candidate, a Django case with a working environment and genuine override ambiguity, went unaudited for leakage and fix-site resolution (C17).

The hidden-type effect is one apparatus at one model: fifteen constructed tasks whose type is redacted rather than hidden as real code hides types, so transfer to generated stubs, vendored packages or compiled boundaries is untested. The conditional retrieval policy is established against one form of insufficiency, a delegation to a hoisted helper in the same module; whether it holds when the missing information sits behind an import, a base class or a generated stub is untested, and a policy that keys on the span's shape rather than on its sufficiency would fail there. Repeated in-loop edits would exercise the after-every-edit channel. Rejection precision needs a gate run over drafts other than the defect's own gold.

Real-repository substitution frequency and live-server latency are unmeasured; six four-seed files lack their seed-2/3 shards and no artifact records a server version. Static tooling does not reach dynamic behavior, incorrect annotations, `Any`-heavy boundaries or logic outside the checker. The resolution answer holds only while the agent can cheaply read the type-bearing source, and every workspace tested was small enough that it could.

7. Conclusion

Semantic navigation added little on the dispatch suite tested, because the receiver's type was readable in the source the agent opened; where that type was withheld, lookup helped, but only once the agent was framed to use it. A compact definition span costs less than a read only when it displaces one; pushing, instructing and self-election did not produce that displacement, and training did. A type check at the submission boundary, with a repair-and-resubmit loop, converted accepted bad completions into repaired ones on twelve seeded defect pairs.

References

[1] Andrew Blinn, Xiang Li, June Hyung Kim, and Cyrus Omar. Statically contextualizing large language models with typed holes. Proceedings of the ACM on Programming Languages, 8(OOPSLA2), 2024. arXiv:2409.00921.

[2] Gwihwan Go, Quan Zhang, Chijin Zhou, Zhao Wei, and Yu Jiang. LSPRAG: Lsp-guided rag for language-agnostic real-time unit test generation. arXiv preprint arXiv:2510.22210, 2025.

[3] Junwei Liu, Yixuan Chen, Mingwei Liu, Xin Peng, and Yiling Lou. STALL+: Boosting llm-based repository-level code completion with static analysis. arXiv preprint arXiv:2406.10018, 2024.

[4] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 2011. PMLR v15:627–635.